

Phase 3 Assignment 3B - Backend API Projects

Goal: Master Phase 1, Phase 2, and Phase 3 by building realistic, database-backed FastAPI systems that combine RESTful design, modular architecture, and real persistence. Scope: Only use topics taught in Lessons 1-27. Standard: Noticeably harder than the Phase 1 and Phase 2 assignments. These projects are a mini capstone for everything learned so far. Data now lives in a real database, not JSON files.

Allowed Scope

Use only concepts from Phase 1, Phase 2, and Phase 3:

Phase 1 - Fundamentals - FastAPI app setup and route decorators - Path parameters, query parameters, request bodies - Pydantic models, nested models, `Field()` - Status codes, `HTTPException`, `JSONResponse`, and the injected `Response` object

Phase 2 - Core Features - `response_model`, multiple models per route, separate input/output/update schemas - `field_validator`, `model_validator`, `model_config` - Custom exception handlers and validation error handling - `Depends()`, sub-dependencies, class-based dependencies, `yield` dependencies - Middleware - `APIRouter` with tags and prefixes - Form data and file uploads - Headers and cookies - Static files and templates (only if a project genuinely needs a non-API surface)

Phase 3 - Database Integration - SQLAlchemy 2.0 models, `Mapped/mapped_column`, column types and constraints - Relationships: one-to-many, many-to-many, association objects, self-referential - The database `Session/AsyncSession` as a `Depends()` dependency - Full CRUD through the ORM - `from_attributes=True` (ORM mode) and schema-vs-model separation with `response_model` - Alembic migrations (`init`, `autogenerate`, `upgrade`, `downgrade`) - Async SQLAlchemy (`create_async_engine`, `AsyncSession`, eager loading with `selectinload`) - SQLAlchemy as an alternative to SQLAlchemy + Pydantic - NoSQL: MongoDB via Motor/Beanie, and Redis for caching (from the optional Lesson 27)

Do not use:

- JWT, OAuth2, password hashing, login/session systems, or any real authentication or authorization
- Background tasks, WebSockets, SSE, streaming responses
- Rate limiting, pagination cursors, or third-party caching frameworks beyond Redis
- Testing frameworks as a required deliverable
- Docker, deployment, CI/CD, secret managers, or environment-configuration packages
- Any Phase 4+ feature, even if it would be useful in a real system

Important: These projects need a concept of "who is acting" (a member, a student, a user). Treat identity as a **validated request input only** - for example a required header such as `X-User-Id` that must reference an existing row. This is request context, not security. Do not build real authentication or role-based access control.

Global Rules

- Build backend APIs only. Do not build a frontend.

- Do not provide solutions, code, pseudocode, or implementation hints in your submission notes.
- **Persist all data in a real relational database using SQLAlchemy 2.0 or SQLModel.** JSON-file storage is no longer acceptable for primary data.
- **Use Alembic for all schema creation and changes.** Do not rely on `create_all` as the migration strategy; provide real migration scripts.
- Use `response_model` and separate create/read/update schemas. Never return raw ORM objects without an output schema.
- Model relationships properly with foreign keys and ORM relationships. Every project must include at least one one-to-many and one many-to-many relationship.
- Split code into modules and routers. Keep database setup, models, schemas, dependencies, and routers in separate files.
- Use dependencies for the database session, entity loading, identity headers, and pagination/filter parsing.
- Enforce integrity at the database level (unique constraints, foreign keys, not-null) in addition to Pydantic validation.
- Each project must run with `uvicorn main:app --reload` after `alembic upgrade head`.
- Design every project as if another developer will maintain it after you.

Recommended persistence pattern:

- One database per project (SQLite is fine for local development; the same code should target PostgreSQL by changing only the connection URL).
- `alembic/` directory with real, ordered migration scripts committed to the project.
- Server-generated integer or UUID primary keys. Never trust a client-supplied primary key.
- Output schemas that hide internal columns (soft-delete flags, internal notes, raw file paths, cost/pricing internals).
- Uploaded files stored in a local `uploads/` folder, with only metadata and a public reference persisted in the database.

Recommended evaluation mindset:

- Correct relational modeling and constraint design
- Clean separation between ORM models and API schemas
- Correct, reversible Alembic migrations
- Consistent validation, error handling, and status codes
- Appropriate use of routers, dependencies, and transactions

Project 1 - Community Library and Lending Platform API

Difficulty Level

Hard

Estimated Completion Time

14-18 hours

Project Overview

Build the backend for a community library that manages books, physical copies, authors, members, loans, and reservation holds. This project is your introduction to serious relational modeling: many-to-many authorship, one-to-many copies, and a lending workflow driven by real database constraints instead of hand-written checks.

Problem Statement

A library needs a backend system to:

- Catalog books, their authors, and their categories
- Track multiple physical copies of each book
- Register members with different membership tiers
- Check copies out to members and accept returns
- Calculate overdue fines
- Manage a reservation hold queue when all copies are checked out
- Report on circulation, overdue loans, and popular titles

The system must behave like a real library backend, where availability, borrowing limits, and fines are enforced by data rules, not guesswork.

Functional Requirements

- Create and manage authors, categories, and books
- Associate books with one or more authors (many-to-many)
- Add and manage physical copies of a book
- Register and manage members
- Check out an available copy to a member
- Return a checked-out copy and finalize any fine
- Place, cancel, and fulfill reservation holds
- List and search the catalog with filtering, sorting, and pagination
- Provide circulation and overdue reporting

Non-Functional Requirements

- Use SQLAlchemy 2.0 (sync) or SQLModel with a real database
- Use Alembic migrations for the full schema
- Use modular routers and dependency-injected sessions
- Enforce uniqueness (ISBN, copy barcode, member email) at the database level

API Requirements

- Use routers such as `authors`, `categories`, `books`, `copies`, `members`, `loans`, and `holds`
- Use separate create/read/update schemas with `response_model`
- Use `from_attributes=True` so ORM objects serialize cleanly
- Use a dependency to load and validate a book, copy, member, or loan by id
- Use a dependency for pagination and shared filter parsing
- Expose a book's authors and available-copy count through nested read schemas

Required API Endpoints

Method	Path	Purpose
POST	<code>/authors</code>	Create author
GET	<code>/authors</code>	List/search authors
GET	<code>/authors/{author_id}</code>	Get one author with their books
POST	<code>/categories</code>	Create category
GET	<code>/categories</code>	List categories
POST	<code>/books</code>	Create book (with author links)
GET	<code>/books</code>	List/search/filter books
GET	<code>/books/{book_id}</code>	Get one book with authors and availability
PUT	<code>/books/{book_id}</code>	Replace book details and author links
DELETE	<code>/books/{book_id}</code>	Delete a book when allowed
POST	<code>/books/{book_id}/copies</code>	Add a physical copy
GET	<code>/books/{book_id}/copies</code>	List copies of a book
PATCH	<code>/copies/{copy_id}</code>	Update copy condition or status
POST	<code>/members</code>	Create member
GET	<code>/members</code>	List/search members
GET	<code>/members/{member_id}</code>	Get member with active loans
POST	<code>/loans</code>	Check out a copy to a member
POST	<code>/loans/{loan_id}/return</code>	Return a copy
GET	<code>/loans</code>	List/filter loans

Method	Path	Purpose
POST	/holds	Place a reservation hold
POST	/holds/{hold_id}/cancel	Cancel a hold
GET	/books/{book_id}/holds	List the hold queue for a book
GET	/dashboard/circulation	Circulation and overdue metrics

Request And Response Expectations

Book creation request should include:

- `title`
- `isbn`
- `category_id`
- `author_ids` as a non-empty list
- `published_year`
- `description`

Book read response should include:

- Generated `id`
- Nested author summaries
- Category name
- `total_copies` and `available_copies`
- Exclude internal soft-delete or audit fields

Loan (checkout) request should include:

- `member_id`
- `copy_id`

Loan response should include:

- Generated `id`
- `loaned_at` and computed `due_at`
- `status`
- `fine_amount` (initialized to zero)

List endpoints should support search, filtering, sorting, and pagination with `page`, `limit`, and `total` in list responses.

Validation Requirements

- `title`: 1-200 characters
- `isbn`: validated format, unique across books
- `published_year`: within a sensible range

- `author_ids`: non-empty; every id must reference an existing author
- Copy `barcode`: unique across all copies
- Copy `condition`: one of `new`, `good`, `worn`, `damaged`
- Copy `status`: one of `available`, `on_loan`, `reserved`, `retired`
- Member `email`: valid format, unique
- `membership_type`: one of `standard`, `premium`, `staff`
- Pagination `page` ≥ 1 , `limit` between 1 and 100
- All numeric constraints enforced with `Field()`

Business Rules

- Borrowing limit depends on membership type (for example standard 3, premium 6, staff 10)
- A checkout succeeds only if the book has an `available` copy and the member is under their limit
- Due date is computed by the server based on membership type; clients cannot set it
- Overdue fines accrue per day past the due date and are finalized on return
- A member with unpaid fines above a threshold cannot check out new copies
- A book cannot be deleted while it has copies on loan or active holds
- When all copies are on loan, a member may place a hold; holds form an ordered queue
- Returning the last needed copy should fulfill the next hold in the queue by marking its copy `reserved`
- A copy that is `retired` or `damaged` cannot be checked out

Edge Cases

- Checking out a copy that is already on loan returns `409`
- Checking out for a member who is over the borrowing limit returns `409`
- Returning a loan that is already returned returns `409`
- Creating a book with a duplicate ISBN returns `409`
- Creating a copy with a duplicate barcode returns `409`
- Referencing a missing author, category, member, or copy returns `404`
- Placing a hold on a book that has available copies returns `409`
- Deleting a book with active loans returns `409`
- Invalid enum, integer, or date query values return `422`

Suggested Database Schema

- `authors` (id, name, bio, created_at)
- `categories` (id, name unique)
- `books` (id, title, isbn unique, category_id -> categories.id, published_year, description, is_active)
- `book_authors` (book_id -> books.id, author_id -> authors.id) — many-to-many association
- `copies` (id, book_id -> books.id, barcode unique, condition, status)

- `members` (id, name, email unique, membership_type, joined_at, unpaid_fines)
- `loans` (id, copy_id -> copies.id, member_id -> members.id, loaned_at, due_at, returned_at, status, fine_amount)
- `holds` (id, book_id -> books.id, member_id -> members.id, placed_at, position, status)

Relationships to model:

- Book to Author: many-to-many via `book_authors`
- Book to Category: many-to-one
- Book to Copy: one-to-many
- Member to Loan: one-to-many
- Copy to Loan: one-to-many
- Book to Hold: one-to-many

Expected Folder Structure

```
community_library_api/
  main.py
  database.py
  models.py
  schemas.py
  dependencies.py
  exceptions.py
  routers/
    authors.py
    categories.py
    books.py
    copies.py
    members.py
    loans.py
    holds.py
  alembic.ini
  alembic/
    env.py
    versions/
  seed.py
```

Deliverables

- Runnable FastAPI project backed by a real database
- Alembic migrations that build the full schema from empty
- A seed script or seed migration with at least 8 books, multiple authors, copies, and 5 members
- Separate create/read/update schemas for every resource
- README with setup steps (`alembic upgrade head`, run command) and example requests
- At least 12 documented manual test cases covering success, `404`, `409`, and `422`

Evaluation Criteria

- Correct relational modeling of the many-to-many and one-to-many relationships
- Clean ORM-model vs API-schema separation
- Correct, reversible Alembic migrations
- Accurate lending, fine, and hold-queue logic
- Consistent status codes and error shapes

Bonus Challenges

- Add a `GET /members/{member_id}/history` endpoint listing past loans with computed totals
- Add `GET /books/popular` ranked by loan count using a database aggregation
- Add a soft-delete flag for books and exclude soft-deleted rows from public listings
- Cache the `/dashboard/circulation` response in Redis with a short TTL and invalidate it on checkout or return

Project 2 - Online Learning Platform API

Difficulty Level

Advanced

Estimated Completion Time

16-22 hours

Project Overview

Build the backend for an online course platform where instructors publish structured courses and students enroll, progress through lessons, and leave reviews. This project pushes deeper into relationships (an enrollment is a many-to-many association object carrying its own data), computed aggregates (ratings, completion percentage), and a publishing workflow.

Problem Statement

A learning platform needs a backend to:

- Let instructors create courses organized into modules and lessons
- Let students enroll in published courses
- Track each student's lesson-by-lesson progress
- Collect course reviews and compute average ratings

- Support catalog browsing with filtering, sorting, and pagination
- Report platform and per-course metrics

Functional Requirements

- Create and manage instructors and courses
- Structure a course into ordered modules and lessons
- Publish and unpublish courses through a controlled workflow
- Enroll students into courses and track progress
- Mark individual lessons complete and recompute course progress
- Submit and list course reviews with average rating and count
- Tag courses and filter the catalog by tag, level, price, and rating

Non-Functional Requirements

- Use SQLAlchemy 2.0 (sync) or SQLModel with a real database
- Use Alembic migrations for the full schema and any later changes
- Use nested read schemas and computed fields for aggregates
- Enforce unique enrollment and unique review per student-course pair at the database level

API Requirements

- Use routers such as `instructors`, `courses`, `modules`, `lessons`, `enrollments`, and `reviews`
- Use separate create/read/update schemas with `response_model`
- Read acting identity from headers such as `X-Instructor-Id` or `X-Student-Id`, validated against existing rows (context only, not authentication)
- Use dependencies for session, entity loading, identity headers, and pagination/filtering
- Use nested output schemas to expose course structure and aggregates
- Use custom exception handling for invalid workflow transitions

Required API Endpoints

Method	Path	Purpose
POST	<code>/instructors</code>	Create instructor
GET	<code>/instructors/{instructor_id}</code>	Get instructor with courses
POST	<code>/courses</code>	Create course
GET	<code>/courses</code>	List/search/filter courses
GET	<code>/courses/{course_id}</code>	Get course with modules, lessons, and aggregates

Method	Path	Purpose
PUT	/courses/{course_id}	Replace course details
PATCH	/courses/{course_id}/status	Publish or unpublish course
POST	/courses/{course_id}/modules	Add a module
PUT	/modules/{module_id}	Replace a module
POST	/modules/{module_id}/lessons	Add a lesson
PUT	/lessons/{lesson_id}	Replace a lesson
POST	/enrollments	Enroll a student in a course
GET	/enrollments/{enrollment_id}	Get enrollment with progress
POST	/enrollments/{enrollment_id}/complete-lesson	Mark a lesson complete
GET	/students/{student_id}/enrollments	List a student's enrollments
POST	/courses/{course_id}/reviews	Submit a review
GET	/courses/{course_id}/reviews	List reviews
GET	/dashboard/platform	Platform-wide metrics

Request And Response Expectations

Course creation request should include:

- `title`
- `slug`
- `instructor_id`
- `level`
- `price`
- `summary`
- `tag_ids` (optional list)

Course detail response should include:

- Generated `id` and `status`
- Nested modules, each with ordered lessons
- `total_lessons`, `enrollment_count`, `average_rating`, `review_count`
- Exclude unpublished-only internal fields from public list views

Enrollment request should include:

- `student_id`
- `course_id`

Review request should include:

- `rating`
- `comment`

Validation Requirements

- `title`: 3-150 characters
- `slug`: unique, lowercase, url-safe pattern
- `level`: one of `beginner`, `intermediate`, `advanced`
- `price`: greater than or equal to zero
- `status`: one of `draft`, `published`, `archived`
- Module and lesson `position`: positive integers, unique within their parent
- `content_type` for lessons: one of `video`, `article`, `quiz`
- `rating`: integer from 1 to 5
- `comment`: 3-1000 characters
- Enrollment unique per student-course pair
- Review unique per student-course pair
- Pagination and filter parameters validated by a dependency

Business Rules

- New courses start as `draft` and can only be enrolled in when `published`
- A course cannot be published unless it has at least one module and at least one lesson
- A student cannot enroll in the same course twice
- Only an enrolled student can mark lessons complete or submit a review
- Progress percentage is computed from completed lessons over total lessons
- A review can only be submitted for a course the student is enrolled in
- Average rating and review count must be recomputed whenever reviews change
- Instructor identity header is required to create or modify that instructor's courses
- Unpublishing a course keeps existing enrollments but blocks new ones

Edge Cases

- Enrolling in a missing or unpublished course returns `404` or `409`
- Duplicate enrollment returns `409`
- Duplicate review returns `409`
- Reviewing or completing a lesson without an enrollment returns `403`-style business error via `409` or a custom validation error
- Publishing an empty course returns `409`
- Marking a lesson complete that does not belong to the course returns `409`
- Missing or invalid identity header returns `422` or a custom error

- Invalid enum, rating, or pagination values return **422**

Suggested Database Schema

- **instructors** (id, name, email unique, bio)
- **courses** (id, title, slug unique, instructor_id -> instructors.id, level, price, status, published_at)
- **modules** (id, course_id -> courses.id, title, position)
- **lessons** (id, module_id -> modules.id, title, content_type, duration_minutes, position, is_preview)
- **students** (id, name, email unique)
- **enrollments** (id, student_id -> students.id, course_id -> courses.id, enrolled_at, status, unique(student_id, course_id))
- **lesson_progress** (id, enrollment_id -> enrollments.id, lesson_id -> lessons.id, completed_at, unique(enrollment_id, lesson_id))
- **reviews** (id, course_id -> courses.id, student_id -> students.id, rating, comment, created_at, unique(course_id, student_id))
- **tags** (id, name unique)
- **course_tags** (course_id -> courses.id, tag_id -> tags.id) — many-to-many

Relationships to model:

- Course to Instructor: many-to-one
- Course to Module to Lesson: nested one-to-many
- Student to Course: many-to-many through **enrollments** (association object with extra fields)
- Enrollment to LessonProgress: one-to-many
- Course to Tag: many-to-many
- Course to Review: one-to-many

Expected Folder Structure

```
online_learning_api/  
  main.py  
  database.py  
  models.py  
  schemas.py  
  dependencies.py  
  exceptions.py  
  routers/  
    instructors.py  
    courses.py  
    modules.py  
    lessons.py  
    enrollments.py  
    reviews.py  
  alembic.ini  
  alembic/  
    env.py  
    versions/  
  seed.py
```

Deliverables

- Runnable database-backed FastAPI project
- Alembic migrations building the full schema, including at least one follow-up migration that adds a column
- Seed data with at least 4 instructors, 6 courses, nested modules and lessons, students, and reviews
- Nested read schemas exposing course structure and computed aggregates
- README with setup, workflow examples, and identity-header usage
- At least 12 documented manual test cases

Evaluation Criteria

- Correct association-object modeling for enrollments and progress
- Accurate computed aggregates (progress, average rating, counts)
- Correct publishing and enrollment workflow enforcement
- Clean nested response schemas and internal-field hiding
- Correct, ordered Alembic migrations including a schema change

Bonus Challenges

- Add coupon codes that adjust course price at enrollment time, validated against constraints
- Add `GET /courses/{course_id}/leaderboard` ranking enrolled students by progress

- Add `response_model_exclude_none=True` where partial responses benefit
- Cache the `/courses` catalog listing in Redis with TTL and invalidate on publish or price change

Project 3 - Personal Finance and Budgeting API

Difficulty Level

Very Advanced

Estimated Completion Time

18-24 hours

Project Overview

Build the backend for a personal finance manager where each user tracks accounts, records income and expense transactions, moves money between accounts, sets category budgets, and views spending reports. This project centers on the concept that makes databases essential: **transactions and data integrity**. A transfer must atomically debit one account and credit another, and account balances must always be derivable and consistent.

Problem Statement

A finance app needs a backend to:

- Let each user manage multiple accounts
- Record categorized income and expense transactions
- Transfer money between accounts atomically
- Organize categories hierarchically
- Set and evaluate monthly budgets per category
- Produce spending, income, and net-worth reports over date ranges
- Keep every user's data strictly isolated

Functional Requirements

- Create and manage accounts per user
- Create hierarchical income and expense categories
- Record, update, and delete transactions
- Transfer funds between two accounts as one atomic operation
- Compute account balances and total net worth
- Define monthly budgets and compare them against actual spending

- Store recurring-transaction rules as schedule metadata only
- Filter and search transactions by date range, category, type, and amount

Non-Functional Requirements

- Use SQLAlchemy 2.0 (sync) or SQLModel with a real database
- Use Alembic migrations for the full schema
- Guarantee atomicity for transfers using a single database transaction with commit or rollback
- Ensure strict per-user data isolation through validated identity context

API Requirements

- Use routers such as `accounts`, `categories`, `transactions`, `transfers`, `budgets`, and `reports`
- Read the acting user from a required header such as `X-User-Id`, validated against existing users (context only, not authentication)
- Scope every query to the acting user
- Use separate create/read/update schemas with `response_model`
- Use a dependency to resolve and validate the current user and any owned resource
- Use a dependency for date-range and pagination parsing

Required API Endpoints

Method	Path	Purpose
POST	<code>/accounts</code>	Create account
GET	<code>/accounts</code>	List the user's accounts with balances
GET	<code>/accounts/{account_id}</code>	Get one account with balance
PATCH	<code>/accounts/{account_id}</code>	Update or archive account
POST	<code>/categories</code>	Create category
GET	<code>/categories</code>	List categories as a tree
POST	<code>/transactions</code>	Record a transaction
GET	<code>/transactions</code>	List/filter transactions
GET	<code>/transactions/{transaction_id}</code>	Get one transaction
PUT	<code>/transactions/{transaction_id}</code>	Replace a transaction
DELETE	<code>/transactions/{transaction_id}</code>	Delete a transaction
POST	<code>/transfers</code>	Transfer funds between accounts

Method	Path	Purpose
GET	/transfers	List transfers
POST	/budgets	Create a monthly budget
GET	/budgets	List budgets with actual-vs-limit
POST	/recurring-rules	Create a recurring rule (metadata only)
GET	/reports/spending	Spending grouped by category over a range
GET	/reports/monthly-summary	Income, expense, and net for a month
GET	/reports/net-worth	Total balance across accounts

Request And Response Expectations

Account creation request should include:

- name
- type
- currency
- opening_balance

Transaction creation request should include:

- account_id
- category_id
- type
- amount
- occurred_on
- description
- merchant (optional)

Transfer request should include:

- from_account_id
- to_account_id
- amount
- occurred_on
- note (optional)

Transfer response should include both resulting transaction references and the updated balances, produced atomically.

Report responses should return computed aggregates, never trusting client-provided totals.

Validation Requirements

- `name`: 1-80 characters
- Account `type`: one of `checking`, `savings`, `credit`, `cash`
- `currency`: 3-letter uppercase code
- `amount`: greater than zero
- `occurred_on`: a valid date, not in the future beyond a small tolerance
- Transaction `type`: one of `income`, `expense`
- Category `kind`: one of `income`, `expense`
- Category `parent_id`: optional, must reference an existing category of the same kind and same user
- Budget `period_month`: valid year-month; unique per category per month per user
- Date-range query parameters validated so `start` is not after `end`
- Pagination `page` ≥ 1 , `limit` between 1 and 100

Business Rules

- A transfer must debit the source and credit the destination in one database transaction; if any step fails, the whole transfer rolls back and no balance changes
- Transferring between the same account is rejected
- A transaction's category kind must match the transaction type (an expense needs an expense category)
- Account balance is computed from opening balance plus transactions and transfers; balances are never stored as a mutable field that can drift
- For non-credit accounts, a transfer or expense that would drive the balance below zero is rejected by a business rule
- Deleting or editing a transaction must keep balances and reports consistent
- All resources are owned by a user; acting on another user's resource returns a not-found error to avoid leaking existence
- Budgets compare the sum of matching expense transactions in the month against the limit
- Recurring rules are stored as metadata only and never auto-execute

Edge Cases

- Transferring more than an available balance on a non-credit account returns `409`
- Transferring to the same account returns `409` or `422`
- Referencing another user's account or category returns `404`
- Creating a category with a mismatched-kind parent returns `409`
- Duplicate budget for a category and month returns `409`
- A date range with `start` after `end` returns `422`
- Deleting an account that still has transactions returns `409` unless explicitly archived
- Missing or invalid `X-User-Id` header returns `422` or a custom error
- Simulated mid-transfer failure must leave balances unchanged (atomicity)

Suggested Database Schema

- **users** (id, name, email unique)
- **accounts** (id, user_id -> users.id, name, type, currency, opening_balance, is_archived)
- **categories** (id, user_id -> users.id, name, kind, parent_id -> categories.id nullable)
- **transactions** (id, account_id -> accounts.id, category_id -> categories.id nullable, type, amount, occurred_on, description, merchant, is_cleared)
- **transfers** (id, user_id -> users.id, from_account_id -> accounts.id, to_account_id -> accounts.id, amount, occurred_on, note)
- **budgets** (id, user_id -> users.id, category_id -> categories.id, period_month, limit_amount, unique(user_id, category_id, period_month))
- **recurring_rules** (id, account_id -> accounts.id, category_id -> categories.id, amount, cadence, next_run_on)

Relationships to model:

- User to Account, Category, Transfer, Budget: one-to-many
- Account to Transaction: one-to-many
- Category self-referential: parent to children (one-to-many)
- Transfer references two accounts (two foreign keys to the same table)

Expected Folder Structure

```
personal_finance_api/  
  main.py  
  database.py  
  models.py  
  schemas.py  
  dependencies.py  
  exceptions.py  
  services/  
    balances.py  
    reports.py  
  routers/  
    accounts.py  
    categories.py  
    transactions.py  
    transfers.py  
    budgets.py  
    reports.py  
  alembic.ini  
  alembic/  
    env.py  
    versions/  
  seed.py
```

Deliverables

- Runnable database-backed FastAPI project with strict per-user isolation
- Alembic migrations building the full schema
- Seed data with at least 3 users, multiple accounts, a category tree, and transactions across several months
- Atomic transfer implementation demonstrated in the README
- Report endpoints returning server-computed aggregates
- README explaining the balance model and transaction atomicity
- At least 14 documented manual test cases, including an atomicity and an isolation case

Evaluation Criteria

- Correct atomic transfer using a single commit/rollback transaction
- Consistent, derivable account balances with no drift
- Correct hierarchical category modeling and self-referential relationship
- Strict per-user data isolation
- Accurate budget and report aggregations
- Correct Alembic migrations and constraint design

Bonus Challenges

- Add `GET /reports/cash-flow` returning month-over-month net cash flow
- Add multi-currency awareness by rejecting transfers between accounts of different currencies
- Add a `GET /budgets/alerts` endpoint listing categories that have exceeded their monthly budget
- Cache the monthly-summary report in Redis keyed by user and month, invalidating on any transaction change in that month

Project 4 - Music Streaming Catalog and Playlist API (Async Capstone)

Difficulty Level

Very Advanced

Estimated Completion Time

22-30 hours

Project Overview

Build the backend for a music streaming service that manages artists, albums, tracks, user playlists, follows, likes, and play history, then surfaces trending and personalized views. This is the capstone: it must be built on **async SQLAlchemy** end to end, model several intersecting many-to-many relationships (including an ordered playlist-track association and self-referential user follows), and use **Redis** for fast play counters and cached hot endpoints.

Problem Statement

A streaming platform needs an async backend to:

- Maintain a catalog of artists, albums, and tracks
- Model featured artists on tracks and ordered tracks within playlists
- Let users create playlists, follow artists and other users, and like tracks
- Record plays and expose play counts efficiently
- Compute trending tracks and simple personalized recommendations
- Serve high-read endpoints quickly using caching

Functional Requirements

- Create and manage artists, albums, and tracks
- Model multiple artists per track (featured credits)
- Create playlists and add, remove, and reorder tracks
- Follow and unfollow artists and other users
- Like and unlike tracks
- Record a play event and increment a fast counter
- List trending tracks over a recent window
- Provide a simple recommendation feed based on followed artists and liked tracks
- Serve catalog and profile reads with caching

Non-Functional Requirements

- Use async SQLAlchemy end to end (`create_async_engine`, `AsyncSession`, async endpoints)
- Eager-load relationships with `selectinload` where responses need them
- Use Alembic migrations for the full schema
- Use Redis for play counters and cached hot endpoints, with correct invalidation
- Keep response latency low on read-heavy endpoints

API Requirements

- Use routers such as `artists`, `albums`, `tracks`, `playlists`, `social`, and `discovery`

- Every database call must be awaited; no blocking DB access inside async endpoints
- Read acting user from a required header such as `X-User-Id`, validated against existing users (context only, not authentication)
- Use separate create/read/update schemas with `response_model`
- Use nested read schemas for album-with-tracks and playlist-with-ordered-tracks
- Use dependencies for the async session, identity, entity loading, and pagination

Required API Endpoints

Method	Path	Purpose
POST	<code>/artists</code>	Create artist
GET	<code>/artists/{artist_id}</code>	Get artist with albums
POST	<code>/albums</code>	Create album
GET	<code>/albums/{album_id}</code>	Get album with ordered tracks
POST	<code>/tracks</code>	Create track with artist credits
GET	<code>/tracks</code>	List/search/filter tracks
GET	<code>/tracks/{track_id}</code>	Get track with artists and play count
POST	<code>/playlists</code>	Create playlist
GET	<code>/playlists/{playlist_id}</code>	Get playlist with ordered tracks
POST	<code>/playlists/{playlist_id}/tracks</code>	Add a track at a position
DELETE	<code>/playlists/{playlist_id}/tracks/{track_id}</code>	Remove a track
PATCH	<code>/playlists/{playlist_id}/reorder</code>	Reorder playlist tracks
POST	<code>/artists/{artist_id}/follow</code>	Follow an artist
DELETE	<code>/artists/{artist_id}/follow</code>	Unfollow an artist
POST	<code>/users/{user_id}/follow</code>	Follow another user
POST	<code>/tracks/{track_id}/like</code>	Like a track
DELETE	<code>/tracks/{track_id}/like</code>	Unlike a track
POST	<code>/tracks/{track_id}/play</code>	Record a play
GET	<code>/discovery/trending</code>	Trending tracks over a window
GET	<code>/discovery/for-you</code>	Personalized recommendations

Method	Path	Purpose
GET	/users/{user_id}/library	The user's playlists, likes, and follows

Request And Response Expectations

Track creation request should include:

- `title`
- `album_id`
- `duration_seconds`
- `track_number`
- `explicit`
- `artist_credits` as a non-empty list of artist ids with a role

Track read response should include:

- Generated `id`
- Nested artist credits
- `play_count` (served from the fast counter where appropriate)

Playlist detail response should include:

- Ordered tracks with their positions
- `total_duration_seconds`
- Owner reference

Add-track-to-playlist request should include:

- `track_id`
- optional `position`

Validation Requirements

- `title`: 1-200 characters
- `duration_seconds`: greater than zero
- `track_number`: positive integer, unique within its album
- `album_type`: one of `album`, `single`, `ep`
- `artist_credits`: non-empty; each id must reference an existing artist; `role` is one of `primary`, `featured`
- Playlist `name`: 1-100 characters
- Playlist position: positive; reorder input must be a valid permutation of the playlist's current tracks
- A track cannot be added to the same playlist twice
- A user cannot follow themselves
- Pagination and trending-window parameters validated by a dependency

Business Rules

- A track belongs to exactly one album but may credit multiple artists
- Playlist tracks are ordered; adding at a position shifts later tracks, and reorder must preserve the exact set of tracks
- Following, liking, and playlist membership are idempotent at the data level (no duplicate rows)
- A play event both records history and increments a Redis counter; the track's `play_count` reflects that counter
- Trending is computed from recent play events over a configurable window
- The personalized feed is derived from the acting user's followed artists and liked tracks; it must not require any Phase 4 feature
- Private playlists are visible only to their owner, enforced by validated identity context
- Cached read endpoints must be invalidated when their underlying data changes

Edge Cases

- Creating a track with a duplicate track number in the same album returns `409`
- Adding a track already in the playlist returns `409`
- Reordering with a set that does not match the playlist's tracks returns `422`
- Following yourself returns `409` or `422`
- Liking an already-liked track is idempotent and does not create duplicates
- Accessing a private playlist as a non-owner returns `404`
- Referencing a missing artist, album, track, or user returns `404`
- Any blocking or non-awaited database access is considered a defect
- Recording a play for a missing track returns `404`

Suggested Database Schema

- `artists` (id, name, is_verified, bio)
- `albums` (id, title, artist_id -> artists.id, release_date, album_type)
- `tracks` (id, title, album_id -> albums.id, duration_seconds, track_number, explicit, unique(album_id, track_number))
- `track_artists` (track_id -> tracks.id, artist_id -> artists.id, role) — many-to-many association with a role
- `users` (id, name, email unique)
- `playlists` (id, user_id -> users.id, name, description, is_public)
- `playlist_tracks` (playlist_id -> playlists.id, track_id -> tracks.id, position, added_at, unique(playlist_id, track_id)) — ordered association object
- `artist_follows` (user_id -> users.id, artist_id -> artists.id) — many-to-many
- `user_follows` (follower_id -> users.id, followee_id -> users.id) — self-referential many-to-many
- `track_likes` (user_id -> users.id, track_id -> tracks.id) — many-to-many
- `listening_history` (id, user_id -> users.id, track_id -> tracks.id, played_at)

Relationships to model:

- Album to Artist: many-to-one
- Album to Track: one-to-many
- Track to Artist: many-to-many via `track_artists` with a role
- Playlist to Track: ordered many-to-many via `playlist_tracks`
- User to User: self-referential many-to-many via `user_follows`
- User to Artist and User to Track: many-to-many via follows and likes

Expected Folder Structure

```
music_streaming_api/  
  main.py  
  database.py  
  cache.py  
  models.py  
  schemas.py  
  dependencies.py  
  exceptions.py  
  services/  
    trending.py  
    recommendations.py  
  routers/  
    artists.py  
    albums.py  
    tracks.py  
    playlists.py  
    social.py  
    discovery.py  
  alembic.ini  
  alembic/  
    env.py  
    versions/  
  seed.py
```

Deliverables

- Runnable async FastAPI project backed by async SQLAlchemy
- Alembic migrations building the full schema
- Redis-backed play counters and at least one cached hot endpoint with correct invalidation
- Seed data with multiple artists, albums, tracks, users, and playlists
- Nested async responses using `selectinload` for relationships
- README covering async setup, Redis usage, and the trending and recommendation logic
- At least 15 documented manual test cases, including ordering, idempotency, and privacy cases

Evaluation Criteria

- Correct, fully async database access with proper eager loading
- Correct modeling of ordered and self-referential many-to-many relationships
- Correct and consistent Redis counters and cache invalidation
- Accurate trending and recommendation computations
- Clean nested schemas and privacy enforcement
- Correct Alembic migrations and constraint design

Bonus Challenges

- Store `listening_history` in MongoDB as documents while keeping the catalog in SQL, and build trending from the document store
- Add `GET /artists/{artist_id}/top-tracks` ranked by play count
- Add a `GET /users/{user_id}/feed` of new releases from followed artists
- Add a Redis-backed rolling daily play count per track and expose a `GET /discovery/rising` endpoint

Final Submission Checklist

For each project, submit:

- Full project folder with modular structure
- `main.py`, `database.py`, `models.py`, `schemas.py`
- Router modules and dependency module
- Exception-handling module if used
- `alembic/` directory with real, ordered migration scripts
- A seed script or seed data
- README with setup steps, example requests, and workflow explanation

Before submitting, verify:

- The database schema is created entirely by Alembic migrations (`alembic upgrade head` on an empty database works)
- At least one migration downgrades cleanly
- The app starts with `uvicorn main:app --reload` and `/docs` shows all routes
- Every resource uses separate create/read/update schemas
- ORM models are never returned directly without an output schema
- Relationships (one-to-many and many-to-many) are modeled with real foreign keys
- Unique and foreign-key constraints are enforced at the database level
- Request validation, business rules, and status codes behave as specified
- Per-user or per-owner data isolation works where required
- No Phase 4+ topic (real auth, background tasks, websockets, streaming) was used

Difficulty Progression

Project	Difficulty	Main Focus
Community Library and Lending Platform API	Hard	Relational modeling, many-to-many authorship, lending and fine logic, Alembic
Online Learning Platform API	Advanced	Association objects, nested aggregates, publishing and enrollment workflow
Personal Finance and Budgeting API	Very Advanced	Atomic transfers, derivable balances, self-referential categories, per-user isolation
Music Streaming Catalog and Playlist API	Very Advanced	Async SQLAlchemy, ordered and self-referential many-to-many, Redis counters and caching

Complete the projects in order. Each project assumes you can already combine Phase 1 fundamentals and Phase 2 architecture with the database skills from Phase 3. By the end, you will have built four production-shaped, database-backed backends that together exercise everything from Lessons 1-27.